

UNIVERSITATEA „ȘTEFAN CEL MARE” DIN SUCEAVA

Facultatea de Inginerie Electrică și Știința Calculatoarelor

Specializarea: Calculatoare, Grupa 3121B

DOCUMENTAȚIE PROIECT

Sistem de Rezervări pentru un Cinema

Disciplina: Programare Orientată pe Obiecte

Student: Balmoș Damaris

Îndrumător: Prof. Prodan Remus

Anul universitar: 2025–2026

1. Introducere

În cadrul acestui proiect am realizat o aplicație în C++ pentru gestionarea rezervărilor de locuri la un cinematograful. Scopul proiectului a fost să pun în practică noțiunile învățate la laboratorul de Programare Orientată pe Obiecte (POO) în acest semestru: crearea de clase, încapsularea datelor, lucrul cu vectori din STL, moștenirea și polimorfismul.

Aplicația este organizată pe mai multe fișiere separate pentru a avea un cod curat și ușor de urmărit. Pe lângă cerințele de bază, am adăugat o interfață (ICinemaService) pentru a folosi funcții virtuale pure și clasa RezervareOnline pentru a demonstra moștenirea. Proiectul a fost urcat pe Git pe un branch numit develop.

2. Descrierea problemei

Aplicația simulează ce se întâmplă în spatele unui sistem simplu de cinema. Aceasta permite:

- Adăugarea și afișarea filmelor din program (titlu, gen, durată, dacă e 2D sau 3D și preț de bază).
- Gestiunea sălilor de cinema folosind o matrice dinamică (`std::vector` de `std::vector`) pentru a vedea care locuri sunt libere [] și care sunt ocupate [X].
- Efectuarea unei rezervări prin selectarea unui rând și a unei coloane. Dacă locul e deja luat sau coordonatele sunt greșite, programul avertizează utilizatorul.
- Calcularea automată a biletului în funcție de tipul filmului (filmele 3D costă cu 10 lei mai mult) și de ziua săptămânii (în weekend se adaugă 5 lei).

3. Structura proiectului și organizarea fișierelor

Codul este împărțit în mai multe module (fișiere .h pentru declarații și .cpp pentru implementare), structurate astfel:

Nume Fișiere	Descriere componentă în proiect
Film.h / Film.cpp	Reține datele unui film (titlu, gen, durată, pretBaza) și calculează prețul final în funcție de zi.
Sala.h / Sala.cpp	Gândește sala ca pe o matrice de numere întregi (0 pentru liber, 1 pentru ocupat) și validează indicii.
Rezervare.h / Rezervare.cpp	Face legătura directă între un obiect de tip Film și un loc specific (rând, coloană) dintr-o Sală.
RezervareOnline.h / RezervareOnline.cpp	Moștenește clasa Rezervare și adaugă ca atribut specific adresa de email a clientului.

ICinemaService.h	Interfața abstractă care impune funcțiile virtuale pure (afisareFilme, afisareLocuri, realizeazaRezervare).
Cinematograf.h / Cinematograf.cpp	Clasa principală de servicii care moștenește ICinemaService și agregă liste de Săli și Filme.
main.cpp	Punctul de pornire al programului. Implementează un meniu interactiv (while-switch) pentru consolă.

4. Analiza claselor și a relațiilor dintre ele

Arhitectura aplicației se bazează pe trei tipuri majore de relații între obiecte:

- Agregare: Clasa Cinematograf conține vectori de obiecte de tip Film și Sala. Colecțiile sunt gestionate de cinematograf, dar obiectele pot exista și independent.
- Asociere: Clasa Rezervare folosește referințe/obiecte de tip Film și Sala pentru a ști cu exactitate ce film și ce scaun fac obiectul tranzacției.
- Moștenire (Is-A): RezervareOnline moștenește public clasa Rezervare, preluând toate caracteristicile acesteia și extinzând funcționalitatea prin câmpul emailClient. De asemenea, Cinematograf moștenește ICinemaService.

5. Concepte de Programare Orientată pe Obiecte aplicate

- Încapsulare: Atributele tuturor claselor sunt declarate în zona private (ex: pretBaza, locuri, emailClient). Modificarea sau citirea lor directă din exterior este interzisă, accesul fiind permis exclusiv prin metode publice de tip getter (cum ar fi getId(), getTitlu()).
- Polimorfism: Interfața ICinemaService definește comportamentul abstract prin funcții virtuale pure (= 0). Implementarea efectivă se face în clasa derivată Cinematograf, separând complet definiția serviciilor de logica lor internă.
- Reutilizarea codului: În constructorul clasei derivate RezervareOnline, atributele comune sunt transmise direct către constructorul clasei părinte folosind lista de inițializare: ': Rezervare(f, s, r, c)'.

6. Tratarea excepțiilor

Pentru a preveni blocarea sau oprirea bruscă a programului în cazul unor date de intrare incorecte, am implementat un mecanism robust de gestionare a erorilor folosind try-catch. În metoda ocupaLoc din clasa Sala, se validează dacă coordonatele rândului și coloanei se află în interiorul matricei. Dacă indicii sunt în afara limitelor, se aruncă o excepție standard:

```
throw out_of_range("Index invalid! Randul sau coloana nu exista.");
```

În cazul în care locul selectat a fost deja rezervat (valoarea sa în matrice este 1), se aruncă o eroare de rulare:

```
throw runtime_error("Locul este deja ocupat!");
```

Toate aceste excepții sunt capturate în main.cpp în interiorul structurii switch a meniului. Mesajul de eroare este afișat curat pe ecran, iar utilizatorul este trimis înapoi în meniu, aplicația continuând să ruleze fără crash.

7. Rezultate și rularea programului

Meniul interactiv creat în consolă oferă o experiență structurată și ușor de testat în timpul prezentării live. Utilizatorul poate afișa dinamic lista filmelor (cu prețurile de bază), poate vizualiza harta grafică a sălii unde scaunele libere apar ca [] și cele ocupate ca [X], și poate efectua rezervări în timp real. Sistemul rulează automat logica de suprataxare pentru filmele 3D sau pentru zilele de weekend în cadrul funcției de calcul al prețului.

8. Instrucțiuni de compilare și rulare

Codul a fost testat și compilat cu succes utilizând setul de instrumente GCC (MinGW). Pentru a compila toate modulele împreună din terminal, navigați în directorul sursă și executați:

```
g++ rezervari_cinema.cpp cinematograf.cpp film.cpp sala.cpp  
rezervare.cpp RezervareOnline.cpp -o app.exe
```

Lansarea în execuție a aplicației se face prin comanda:

```
./app.exe
```

9. Concluzii

Realizarea acestui proiect a reprezentat o oportunitate excelentă de a înțelege cum interacționează mai multe clase într-o aplicație C++ completă. Tranziția de la exemplele teoretice din curs la scrierea unui sistem funcțional cu logică de business (atribuiri de locuri, matrice dinamice, moșteniri) mi-a clarificat importanța utilizării corecte a listelor de inițializare și a tehnicilor de încapsulare.

Cea mai mare provocare întâmpinată a fost legată de lipsa unui constructor implicit (default) la clasele compuse, problemă care a fost corectată prin transmiterea corectă a obiectelor în constructorii claselor Rezervare și RezervareOnline. Aplicația îndeplinește criteriile din baremul temei 3121B și este pregătită pentru evaluarea automată și prezentarea live.